

Especificación de Requisitos de Software (SRS)

SGROAS — v0.9.0-rc

Equipo CET

Universidad Técnica Estatal de Quevedo — Carrera de Ingeniería de Software

30 de julio de 2026

Índice

1. Introducción	3
1.1. Propósito	3
1.2. Alcance	3
1.3. Definiciones y abreviaturas	3
1.4. Referencias	4
1.5. Resumen del documento	4
2. Descripción global	4
2.1. Perspectiva del producto	4
2.2. Funciones del producto	5
2.3. Características de los usuarios	5
2.4. Restricciones	5
2.5. Supuestos y dependencias	5
3. Requisitos específicos	6
3.1. Convenciones	6
3.2. Requisitos funcionales	6
3.2.1. REQ-F-001 — Inicio de sesión	6
3.2.2. REQ-F-002 — Registro de usuario	6
3.2.3. REQ-F-003 — Renovación de token	7

3.2.4.	REQ-F-004 — Cierre de sesión	7
3.2.5.	REQ-F-005 a REQ-F-009 — CRUD de conductores	7
3.2.6.	REQ-F-010 a REQ-F-014 — CRUD de usuarios	8
3.3.	Requisitos no funcionales	8
3.3.1.	REQ-NF-001 — Cabeceras de seguridad HTTP	8
3.3.2.	REQ-NF-002 — Cifrado en tránsito	9
3.3.3.	REQ-NF-003 — Rendimiento del listado	9
3.3.4.	REQ-NF-004 — Protección contra inyección	9
3.3.5.	REQ-NF-005 — Limitación de intentos de login	10
3.3.6.	REQ-NF-006 — Cobertura de pruebas	10
3.4.	Requisitos de interfaz externa	10
3.5.	Matriz de trazabilidad	11
3.6.	Gestión del cambio	11
Declaración de conformidad		11

1. Introducción

1.1. Propósito

El presente documento constituye la Especificación de Requisitos de Software (SRS) del sistema **SGROAS** (Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad), elaborada conforme a la especificación internacional **ISO/IEC/IEEE 29148:2018** [1]. Define los requisitos funcionales y no funcionales del sistema en su estado candidato **v0.9.0-rc** (Tercera Entrega del Proyecto Fin de Curso, PFC), su prioridad según el método MoSCoW, sus criterios de aceptación medibles y su método de verificación.

1.2. Alcance

El sistema SGROAS gestiona los recursos operativos, administrativos y de seguridad de una institución de transporte. En el estado **v0.9.0-rc** el alcance comprende: autenticación stateless mediante JWT en *cookie* HttpOnly, CRUD de conductores y de usuarios, caché Redis, procedimientos almacenados para reportes y agregaciones, y la consolidación funcional heredada de la Entrega 1B. Quedan fuera del alcance de esta entrega los módulos de facturación, pagos y la puesta en producción pública (prevista para la Entrega Final, **v1.0.0**).

1.3. Definiciones y abreviaturas

SGROAS Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad.

JWT JSON Web Token, estándar RFC 7519 de representación segura de afirmaciones.

CRUD Create, Read, Update, Delete (crear, leer, actualizar, eliminar).

MoSCoW Técnica de priorización: Must (obligatorio), Should (debería), Could (podría), Won't (no se hará).

ORM Mapeo objeto-relacional (Hibernate/Spring Data JPA).

SP *Stored procedure* (procedimiento almacenado) o función SQL.

RNF Requisito no funcional.

RF Requisito funcional.

TTL *Time to Live*, duración máxima de una entrada de caché.

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — Ingeniería de requisitos [1].
- INCOSE Guide to Writing Requirements v4 [2].
- Cohn, M. *User Stories Applied* (criterios INVEST) [3].
- Cockburn, A. *Writing Effective Use Cases* [4].
- RFC 7519 — JSON Web Token [5].
- RFC 7807 — Problem Details for HTTP APIs [6].
- OWASP Top 10:2021 [7].
- Robertson y Robertson. *Mastering the Requirements Process* (plantilla Volere) [8].
- Pohl, K. *Requirements Engineering* [9].
- Wiegers y Beatty. *Software Requirements* [10].
- ISO/IEC 25010:2011 — Calidad del producto software [11].

1.5. Resumen del documento

El documento se organiza en tres capítulos: la presente introducción, la descripción global del producto (perspectiva, funciones, usuarios, restricciones, supuestos) y el capítulo de requisitos específicos (funcionales, no funcionales y de interfaz externa). Cada requisito indica su identificador persistente, *rationale*, prioridad MoSCoW, criterio de aceptación medible, método de verificación y trazabilidad hacia historia de usuario, caso de uso, módulo, endpoint y evidencia empírica, en correspondencia con `docs/trazabilidad/matriz.csv`.

2. Descripción global

2.1. Perspectiva del producto

SGROAS es una aplicación web de tres capas: frontend Angular 20, backend Spring Boot 3.2.x (Java 21 LTS) y PostgreSQL 16, con Redis 7 como caché distribuida y orquestación mediante Docker Compose. El backend expone una API REST documentada en OpenAPI 3.0 en `/api/docs`. La arquitectura se detalla en los diagramas C4 (niveles 1–3) en `docs/arquitectura/` y en las decisiones registradas en `docs/adr/`.

2.2. Funciones del producto

- Autenticación stateless con JWT en *cookie* HttpOnly + Secure + SameSite=Strict.
- CRUD de conductores (alta, consulta, edición, eliminación lógica).
- CRUD de usuarios del sistema (alta, consulta, edición, eliminación lógica).
- Caché Redis en el listado paginado de conductores con TTL configurable externamente.
- Reportes y agregaciones mediante procedimientos almacenados y funciones SQL.
- Manejo uniforme de errores con ProblemDetails (RFC 7807).

2.3. Características de los usuarios

Rol	Permiso	Descripción
Administrador	Total	Gestiona usuarios, conductores y configuración del sistema.
Coordinador	Operativo	Gestiona conductores, rutas, vehículos y asignaciones.
Seguridad	Consulta	Consulta incidentes, reportes y estado de la flota.

2.4. Restricciones

- El esquema de base de datos se aplica exclusivamente desde `db/schema.sql`, `db/seed.sql` y migraciones Flyway; queda prohibido `ddl-auto=update`.
- Toda operación que no sea CRUD elemental se implementa como procedimiento almacenado o función SQL (Bloque A.2.2 de la guía).
- Queda prohibida la concatenación de entrada de usuario en cualquier consulta SQL, JPQL o HQL, y el uso de SQL dinámico en procedimientos.
- Las imágenes de contenedor se fijan por *digest* sha256.

2.5. Supuestos y dependencias

- Se dispone de Docker Desktop para la orquestación y de Java 21 para la compilación.
- Las credenciales semilla (`admin@sgroas.com` / `admin123`) se documentan en el README y en `db/seed.sql`.

- La ejecución de las mediciones (k6, Lighthouse, JaCoCo) requiere el sistema levantado según el Makefile.

3. Requisitos específicos

3.1. Convenciones

Cada requisito declara: **ID** (identificador persistente), **Tipo** (funcional / no funcional), **Prioridad** (MoSCoW), **Rationale** (justificación y origen), **Criterio de aceptación** (medible), **Método de verificación** (*inspection, analysis, demonstration, test*) y **Trazabilidad** (historia, caso de uso, módulo, endpoint, prueba, evidencia). Los identificadores coinciden con la matriz docs/trazabilidad/matriz.csv.

3.2. Requisitos funcionales

3.2.1. REQ-F-001 — Inicio de sesión

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** El acceso al sistema requiere autenticación previa (origen: entrevistas con el personal de transporte).
- **Enunciado:** El sistema deberá autenticar al usuario mediante `POST /api/auth/login` y, tras validar credenciales, emitir un JWT firmado y establecer una *cookie* `HttpOnly + Secure + SameSite=Strict` (RFC 7519).
- **Criterio de aceptación:** Con credenciales válidas responde `200 OK` y *cookie* con el JWT; con credenciales inválidas responde `401 Unauthorized` con `ProblemDetails`.
- **Método de verificación:** *test* (`AuthServiceTest`) y *demonstration* (Postman).
- **Trazabilidad:** HU-001, CU-001, `AuthController`, `POST /api/auth/login`, evidencia docs/mediciones/sec/.

3.2.2. REQ-F-002 — Registro de usuario

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** El administrador debe poder crear cuentas de acceso.
- **Enunciado:** El sistema deberá permitir registrar un usuario mediante `POST /api/auth/register` validando unicidad de correo.

- **Criterio de aceptación:** Registro exitoso responde 201 `Created`; correo duplicado responde 409 `Conflict`.
- **Método de verificación:** *test* (`AuthServiceTest`).
- **Trazabilidad:** HU-001, `AuthController`, `POST /api/auth/register`.

3.2.3. REQ-F-003 — Renovación de token

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** Evitar reautenticación frecuente manteniendo la seguridad.
- **Enunciado:** El sistema deberá renovar el JWT mediante `POST /api/auth/refresh` verificando el *refresh token*.
- **Criterio de aceptación:** *Refresh token* válido responde 200 `OK` con nuevo JWT; inválido o revocado responde 401.
- **Método de verificación:** *test* (`AuthServiceTest`).
- **Trazabilidad:** HU-001, `AuthController`, `POST /api/auth/refresh`.

3.2.4. REQ-F-004 — Cierre de sesión

- **Tipo:** Funcional. **Prioridad:** Must.
- **Rationale:** Permitir revocar la sesión activa.
- **Enunciado:** El sistema deberá revocar el token (JTI en lista negra Redis) mediante `POST /api/auth/logout`.
- **Criterio de aceptación:** Responde 204 `No Content` y el token queda revocado en Redis.
- **Método de verificación:** *test* (`AuthServiceTest`).
- **Trazabilidad:** HU-001, `AuthController`, `POST /api/auth/logout`.

3.2.5. REQ-F-005 a REQ-F-009 — CRUD de conductores

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** La gestión de la flota requiere el mantenimiento de conductores (origen: REQ-005 de la Entrega 1A).
- **Enunciados:**

REQ-F-005 Listado paginado y filtrable: GET /api/conductores.

REQ-F-006 Consulta por id: GET /api/conductores/{id}.

REQ-F-007 Alta: POST /api/conductores.

REQ-F-008 Actualización: PUT /api/conductores/{id}.

REQ-F-009 Eliminación lógica: DELETE /api/conductores/{id}.

- **Criterio de aceptación:** Cada operación responde según REST (200, 201, 204); la cédula es única; la eliminación cambia el estado sin borrar la fila.
- **Método de verificación:** *test* (ConductorServiceTest) y *demonstration* (Postman).
- **Trazabilidad:** HU-002, CU-002, ConductorController, evidencia docs/mediciones/perf/ (REQ-F-005).

3.2.6. REQ-F-010 a REQ-F-014 — CRUD de usuarios

- **Tipo:** Funcional. **Prioridad:** Must (cada uno).
- **Rationale:** El administrador debe gestionar las cuentas del sistema.
- **Enunciados:**

REQ-F-010 Listado paginado: GET /api/usuarios.

REQ-F-011 Consulta por id: GET /api/usuarios/{id}.

REQ-F-012 Alta: POST /api/usuarios.

REQ-F-013 Actualización: PUT /api/usuarios/{id}.

REQ-F-014 Eliminación lógica: DELETE /api/usuarios/{id}.

- **Criterio de aceptación:** Respuestas REST correctas; el rol se restringe a los enumerados (ADMIN, COORDINADOR, SEGURIDAD).
- **Método de verificación:** *test* (UsuarioController/Service) y *demonstration*.
- **Trazabilidad:** HU-003, CU-003, UsuarioController.

3.3. Requisitos no funcionales

3.3.1. REQ-NF-001 — Cabeceras de seguridad HTTP

- **Tipo:** No funcional. **Prioridad:** Must.
- **Rationale:** Mitigar ataques de malconfiguración (OWASP A05).

- **Enunciado:** Toda respuesta HTTP incluirá `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` y `Content-Security-Policy`.
- **Criterio de aceptación:** `curl -I` contra la raíz muestra las cuatro cabeceras.
- **Método de verificación:** *test y demonstration* (docs/mediciones/sec/A05-*).
Estado: verificado.

3.3.2. REQ-NF-002 — Cifrado en tránsito

- **Tipo:** No funcional. **Prioridad:** Must.
- **Rationale:** Confidencialidad e integridad (OWASP A02).
- **Enunciado:** La conexión utilizará TLS v1.3 con suites AEAD.
- **Criterio de aceptación:** `curl -v` muestra TLSv1.3 y suite TLS_AES_256_GCM_SHA384.
- **Método de verificación:** *demonstration* (docs/mediciones/sec/A02-*). **Estado:** verificado.

3.3.3. REQ-NF-003 — Rendimiento del listado

- **Tipo:** No funcional. **Prioridad:** Must.
- **Rationale:** El listado de conductores es la consulta más frecuente (ISO/IEC 25010, eficiencia de desempeño).
- **Enunciado:** El `GET /api/conductores` con caché caliente responderá con p95 menor a 200 ms.
- **Criterio de aceptación:** Tres corridas k6 (50 VUs, 30 s) con p95 medio de 81,43 ms y 0 % de errores.
- **Método de verificación:** *test* (k6), reporte docs/mediciones/perf/REPORT.md.
Estado: verificado.

3.3.4. REQ-NF-004 — Protección contra inyección

- **Tipo:** No funcional. **Prioridad:** Must.
- **Rationale:** Evitar inyección SQL (OWASP A03).
- **Enunciado:** Los payloads de entrada se parametrizan estrictamente; los procedimientos almacenados no construyen SQL dinámico.

- **Criterio de aceptación:** POST con payload ' OR '1'='1 responde 422 con ProblemDetails; el auditor scripts/audit-sql-dynamic.sh no reporta violaciones.
- **Método de verificación:** *test y analysis* (docs/mediciones/sec/A03-*). **Estado:** verificado.

3.3.5. REQ-NF-005 — Limitación de intentos de login

- **Tipo:** No funcional. **Prioridad:** Must.
- **Rationale:** Mitigar fuerza bruta (OWASP A07).
- **Enunciado:** Seis intentos fallidos consecutivos desde la misma IP responderán 429 Too Many Requests.
- **Criterio de aceptación:** La séptima petición (y siguientes) responde 429.
- **Método de verificación:** *demonstration* (docs/mediciones/sec/A07-*). **Estado:** verificado.

3.3.6. REQ-NF-006 — Cobertura de pruebas

- **Tipo:** No funcional. **Prioridad:** Should (objetivo 60 % para la Tercera Entrega, 70 % para la Final).
- **Rationale:** Sostener la calidad y la tendencia creciente entre entregas.
- **Enunciado:** El reporte JaCoCo sobre el módulo de dominio y servicios alcanzará al menos 60 % de instrucciones en la Tercera Entrega.
- **Criterio de aceptación:** jacoco.xml con cobertura ≥ 60 %.
- **Método de verificación:** *test* (docs/mediciones/jacoco/). **Estado:** verificado (98,8 % de instrucciones y 85,4 % de ramas con 150 pruebas JUnit 5; umbral check de 60 % activo en mvn verify).

3.4. Requisitos de interfaz externa

- **REQ-IF-001** (Must): La API REST se documentará en OpenAPI 3.0 accesible en /api/docs. Verificación: *inspection* (Swagger UI, Figura 2 del informe).
- **REQ-IF-002** (Must): Los errores seguirán el formato ProblemDetails (RFC 7807) con type, title, status, detail e instance. Verificación: *test y demonstration* (Postman).
- **REQ-IF-003** (Should): El frontend se comunicará con el backend mediante *cookie* HttpOnly (sin almacenar el JWT en localStorage).

3.5. Matriz de trazabilidad

La matriz end-to-end se versiona en `docs/trazabilidad/matriz.csv` con las columnas `id_requisito`, `tipo`, `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso`, `evidencia_empirica` y `estado`. Cubre el 100 % de los requisitos Must (14 funcionales + 5 no funcionales) vigentes en `v0.9.0-rc` y su validez se comprueba en CI con `scripts/validate-traceability.sh`.

3.6. Gestión del cambio

Las modificaciones a los requisitos se registran en `docs/requisitos/CHANGELOG-REQ.md` siguiendo *Keep a Changelog*; las decisiones de diseño que afectan requisitos Must se documentan como ADR en `docs/adr/` (plantilla Nygard).

Declaración de conformidad

Cada requisito de esta especificación se redactó verificando las nueve características de calidad para requisitos individuales y las seis para conjuntos de requisitos del INCOSE Guide to Writing Requirements v4 [2]: Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming (C1–C9) y Complete, Consistent, Feasible, Comprehensible, Able to be validated, Correct (C10–C15).

Referencias

- [1] International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, “ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering,” ISO/IEC/IEEE, Technical Report, 2018.
- [2] International Council on Systems Engineering, “INCOSE Guide to Writing Requirements,” INCOSE — Requirements Working Group, Technical Report INCOSE-TP-2010-006-04, 2023.
- [3] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA: Addison-Wesley Professional, 2004.
- [4] A. Cockburn, *Writing Effective Use Cases*. Boston, MA: Addison-Wesley Professional, 2001.
- [5] M. Jones, J. Bradley y N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [6] M. Nottingham y E. Wilde, *Problem Details for HTTP APIs*, RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [7] OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, 2021.
- [8] S. Robertson y J. Robertson, *Mastering the Requirements Process: Getting Requirements Right*, 3.^a ed. Boston, MA: Addison-Wesley Professional, 2012.
- [9] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Heidelberg: Springer, 2010.
- [10] K. E. Wiegers y J. Beatty, *Software Requirements*, 3.^a ed. Redmond, WA: Microsoft Press, 2013.
- [11] ISO/IEC, *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE)*, 2011.